



Guide Pratique

Git & GitHub

De zéro à la maîtrise complète

Toutes les commandes

Exercices pratiques

Conseils & Astuces



2026 — Formation Git Complète



Table des matières

1. Premiers pas — Installation & Configuration

2. Créer et gérer un dépôt

3. Sauvegarder son travail — add, commit, log

4. Corriger des erreurs — diff, restore, reset

5. Travailler avec les branches

6. Collaborer avec GitHub — push, pull, clone

7. Commandes avancées — stash, tag, rebase

8. Mémo récapitulatif complet

9. Exercices pratiques guidés

1. Premiers pas — Installation & Configuration

1.1 — Installer Git

Git s'installe différemment selon votre système d'exploitation :

Système	Méthode d'installation
Windows	Télécharger sur git-scm.com → Installer avec options par défaut
macOS	Terminal : <code>xcode-select --install</code> ou <code>brew install git</code>
Linux (Ubuntu)	Terminal : <code>sudo apt install git</code>

Vérifier l'installation :

```
# Vérifier que Git est bien installé
git --version

# Résultat attendu :
# git version 2.43.0
```

1.2 — Configurer votre identité (une seule fois)

Avant d'utiliser Git, vous devez vous identifier. Cette information apparaîtra dans chacun de vos commits :

```
# Configurer votre nom
git config --global user.name "Votre Nom"

# Configurer votre email
git config --global user.email "votre@email.com"

# Définir l'éditeur de texte par défaut (optionnel)
git config --global core.editor "notepad"

# Vérifier toute la configuration
git config --list
```

■ Conseil

L'option `--global` s'applique à tous vos projets sur ce PC. Sans `--global`, la configuration ne s'applique qu'au projet courant.

2. Créer et gérer un dépôt

2.1 — Initialiser un nouveau dépôt

Un dépôt (repository) est un dossier dont l'historique est suivi par Git.

```
# Créer un nouveau dossier
mkdir mon-projet

# Entrer dans le dossier
cd mon-projet

# Initialiser Git dans ce dossier
git init

# Résultat attendu :
# Initialized empty Git repository in .../mon-projet/.git/
```

■ ■ Attention

git init crée un dossier caché .git dans votre projet. C'est là que Git stocke tout l'historique. Ne supprimez jamais ce dossier !

2.2 — Vérifier l'état du dépôt

```
# Voir l'état actuel de votre dépôt
git status

# Messages courants :
# On branch master → vous êtes sur la branche principale
# No commits yet → aucun commit encore effectué
# Untracked files → fichiers non encore suivis par Git
# Changes not staged → fichiers modifiés mais non préparés
# Changes to be committed → fichiers prêts à être commités
# nothing to commit → tout est sauvegardé
```

3. Sauvegarder son travail — add, commit, log

Le cycle de travail fondamental de Git :

Chaque modification passe par 3 zones avant d'être sauvegardée :

Zone de travail
(Working Directory)

→ git add →

Zone de
staging (Index)

→ git commit →

Dépôt local
(Repository)

3.1 — git add — Préparer les fichiers

```
# Ajouter un fichier spécifique
git add nom-fichier.txt

# Ajouter tous les fichiers modifiés
git add .

# Ajouter tous les fichiers .R (exemple)
git add *.R

# Ajouter un dossier entier
git add dossier/

# Retirer un fichier du staging (annuler git add)
git restore --staged nom-fichier.txt
```

3.2 — git commit — Sauvegarder

```
# Commit avec un message court
git commit -m "Description de ce que vous avez fait"

# Commit avec un message long (ouvre l'éditeur)
git commit

# Modifier le dernier message de commit
git commit --amend -m "Nouveau message corrigé"

# Commit en ajoutant automatiquement les fichiers suivis
git commit -am "Message"
```

■ Bonne pratique

Un bon message de commit répond à : Quoi ? Pourquoi ? Ex: "Ajout analyse descriptive des données 2024" plutôt que "modification" ou "update".

3.3 — git log — Consulter l'historique

```
# Voir tout l'historique (détaillé)
git log

# Version courte et lisible – RECOMMANDÉE
git log --oneline

# Avec graphique des branches
git log --oneline --graph --all

# Les 5 derniers commits seulement
git log -5

# Chercher dans les messages de commit
git log --grep="mot-clé"

# Voir les commits d'un fichier précis
git log -- nom-fichier.txt
```

4. Corriger des erreurs — diff, restore, reset

4.1 — git diff — Voir les modifications

```
# Voir les modifications non sauvegardées
git diff

# Voir les modifications dans le staging
git diff --staged

# Comparer deux commits
git diff abc1234 def5678

# Comparer deux branches
git diff main ma-branche
```

4.2 — git restore — Annuler des modifications

```
# Annuler les modifications d'un fichier (avant git add)
git restore nom-fichier.txt

# Annuler les modifications de TOUS les fichiers
git restore .

# Retirer un fichier du staging (après git add)
git restore --staged nom-fichier.txt

# Restaurer un fichier depuis un commit précis
git restore --source=abc1234 nom-fichier.txt
```

■ ■ Attention

git restore annule définitivement vos modifications non committées. Cette opération est irréversible ! Utilisez-la avec précaution.

4.3 — git reset — Revenir en arrière

git reset permet de revenir à un état précédent. Il existe 3 modes :

```
# Mode SOFT : garde vos modifications dans le staging
git reset --soft HEAD~1

# Mode MIXED (défaut) : garde vos modifications dans le working dir
git reset HEAD~1

# Mode HARD : supprime TOUTES les modifications (DANGEREUX !)
git reset --hard HEAD~1

# Revenir à un commit précis
git reset --soft abc1234

# HEAD~1 = 1 commit en arrière, HEAD~3 = 3 commits en arrière
```

Mode	Staging	Fichiers	Usage
--soft	Conservé	Conservé	Corriger le message du commit
--mixed	Vidé	Conservé	Dépréparer des fichiers
--hard	Vidé	Supprimé	Annuler complètement (danger !)

4.4 — git revert — Annuler sans effacer l'historique

```
# Créer un commit qui annule le dernier commit
git revert HEAD
# Annuler un commit précis (sans supprimer l'historique)
git revert abc1234
# Différence avec reset :
# revert = ajoute un nouveau commit d'annulation (sûr)
# reset = supprime des commits (risqué si déjà pushé)
```


5. Travailler avec les branches

Une branche est une copie isolée de votre projet. Elle vous permet de tester de nouvelles idées sans risquer de casser le code principal.

■ Conseil

La branche principale s'appelle `main` (ou `master` sur les anciens projets). Ne travaillez jamais directement sur `main` — créez toujours une nouvelle branche !

5.1 — Créer et naviguer entre les branches

```
# Voir toutes les branches
git branch

# Créer une nouvelle branche
git branch nom-branche

# Aller sur une branche
git checkout nom-branche

# Créer ET aller dessus en une seule commande
git checkout -b nom-branche

# Nouvelle syntaxe (Git moderne)
git switch -c nom-branche

# Revenir sur la branche principale
git checkout main
```

5.2 — Fusionner les branches

```
# 1. Aller sur la branche qui reçoit la fusion
git checkout main

# 2. Fusionner une branche dans main
git merge nom-branche

# Fusion avec message de commit personnalisé
git merge nom-branche -m "Fusion de la fonctionnalité X"

# Supprimer une branche après fusion
git branch -d nom-branche

# Forcer la suppression (sans avoir fusionné)
git branch -D nom-branche
```

5.3 — Résoudre les conflits

Un conflit survient quand deux branches ont modifié la même ligne d'un fichier. Git marque les conflits ainsi :

```
# Dans le fichier en conflit, Git affiche :
<<<<<< HEAD
Votre version (branche actuelle)
=====
Leur version (branche fusionnée)
>>>>>> nom-branche

# Pour résoudre :
# 1. Ouvrir le fichier et choisir la bonne version
# 2. Supprimer les marqueurs <<<, ==, >>>
# 3. git add fichier-conflit.txt
# 4. git commit -m "Résolution du conflit"
```

6. Collaborer avec GitHub — push, pull, clone

6.1 — Connecter un dépôt local à GitHub

```
# Ajouter un dépôt distant (après création sur github.com)
git remote add origin https://github.com/pseudo/projet.git

# Vérifier les dépôts distants configurés
git remote -v

# Changer l'URL d'un dépôt distant
git remote set-url origin https://github.com/pseudo/nouveau.git

# Supprimer un dépôt distant
git remote remove origin
```

6.2 — git push — Envoyer vers GitHub

```
# Premier push (définit la branche de suivi)
git push -u origin main

# Push classique (après le premier)
git push

# Push d'une branche spécifique
git push origin nom-branche

# Push de tous les tags
git push --tags
```

6.3 — git pull — Récupérer depuis GitHub

```
# Récupérer ET fusionner les changements
git pull

# Équivalent de pull (en deux étapes)
git fetch origin
git merge origin/main

# Pull avec rebase (historique plus propre)
git pull --rebase

# Voir ce qui a changé sans fusionner
git fetch origin
git log HEAD..origin/main --oneline
```

6.4 — git clone — Copier un projet

```
# Cloner un dépôt GitHub
git clone https://github.com/pseudo/projet.git

# Cloner dans un dossier avec un nom différent
git clone https://github.com/pseudo/projet.git mon-dossier

# Cloner une branche spécifique
git clone -b nom-branche https://github.com/pseudo/projet.git
```

7. Commandes avancées — stash, tag, rebase

7.1 — git stash — Mettre en pause

git stash met de côté vos modifications non committées pour les retrouver plus tard :

```
# Mettre de côté toutes les modifications
git stash

# Avec un message descriptif
git stash save "Travail en cours sur la fonctionnalité X"

# Voir la liste des stashes
git stash list

# Récupérer le dernier stash (et le supprimer)
git stash pop

# Récupérer un stash spécifique
git stash apply stash@{2}

# Supprimer un stash
git stash drop stash@{0}

# Supprimer tous les stashes
git stash clear
```

7.2 — git tag — Marquer des versions

```
# Créer un tag simple
git tag v1.0

# Créer un tag annoté (recommandé)
git tag -a v1.0 -m "Version 1.0 stable"

# Voir tous les tags
git tag

# Voir les détails d'un tag
git show v1.0

# Envoyer les tags sur GitHub
git push origin --tags

# Supprimer un tag
git tag -d v1.0
```

7.3 — .gitignore — Ignorer des fichiers

Le fichier .gitignore liste les fichiers que Git ne doit pas suivre (données sensibles, fichiers générés, etc.) :

```
# Contenu typique d'un .gitignore pour R :  
.Rhistory  
.RData  
.Rproj.user  
*.csv # Ignorer tous les CSV  
data/raw/ # Ignorer un dossier entier  
config.R # Ignorer les fichiers de config  
  
# Créer le fichier .gitignore  
touch .gitignore  
  
# Vérifier ce qui est ignoré  
git status --ignored
```

8. Mémo récapitulatif complet

Configuration

Commande	Description
<code>git config --global user.name "Nom"</code>	Définir votre nom
<code>git config --global user.email "email"</code>	Définir votre email
<code>git config --list</code>	Voir toute la configuration

Démarrer

Commande	Description
<code>git init</code>	Initialiser un dépôt Git
<code>git clone</code>	Copier un dépôt existant
<code>git status</code>	Voir l'état du dépôt

Sauvegarder

Commande	Description
<code>git add .</code>	Préparer tous les fichiers
<code>git add</code>	Préparer un fichier précis
<code>git commit -m "msg"</code>	Créer un commit
<code>git log --oneline</code>	Voir l'historique

Corriger

Commande	Description
<code>git diff</code>	Voir les modifications
<code>git restore</code>	Annuler les modifications
<code>git restore --staged</code>	Retirer du staging
<code>git reset --soft HEAD~1</code>	Défaire le dernier commit
<code>git revert HEAD</code>	Annuler sans effacer

Branches

Commande	Description
<code>git branch</code>	Lister les branches
<code>git checkout -b</code>	Créer et aller sur une branche
<code>git merge</code>	Fusionner une branche
<code>git branch -d</code>	Supprimer une branche

GitHub

Commande	Description
<code>git remote add origin</code>	Connecter à GitHub
<code>git push -u origin main</code>	Premier envoi
<code>git push</code>	Envoyer les commits
<code>git pull</code>	Récupérer les changements

Avancé

Commande	Description
<code>git stash</code>	Mettre en pause
<code>git stash pop</code>	Récupérer le stash
<code>git tag v1.0</code>	Marquer une version
<code>git log --graph --all</code>	Visualiser les branches

9. Exercices pratiques guidés

Exercice

01

Mon premier dépôt Git

■ Objectif : Créer un dépôt local et faire ses premiers commits

#	Commande	Explication
1	<code>mkdir exercice-01 && cd exercice-01</code>	Créer le dossier
2	<code>git init</code>	Initialiser Git
3	<code>echo "Bonjour Git" > notes.txt</code>	Créer un fichier
4	<code>git status</code>	Vérifier l'état
5	<code>git add .</code>	Préparer le fichier
6	<code>git commit -m "Premier commit"</code>	Sauvegarder
7	<code>git log --oneline</code>	Voir l'historique

Exercice

02

Modifier et suivre les changements

■ Objectif : Comprendre le cycle `add` → `commit` avec plusieurs modifications

#	Commande	Explication
1	<code>echo "Ligne 2" >> notes.txt</code>	Ajouter du contenu
2	<code>git diff</code>	Voir les modifications
3	<code>git add notes.txt</code>	Préparer
4	<code>git commit -m "Ajout ligne 2"</code>	Sauvegarder
5	<code>echo "Ligne 3" >> notes.txt</code>	Modifier encore
6	<code>git add . && git commit -m "Ajout ligne 3"</code>	Second commit
7	<code>git log --oneline</code>	Voir les 3 commits

Exercice

03

Travailler avec les branches

■ Objectif : Créer une branche, travailler dessus et la fusionner

#	Commande	Explication
---	----------	-------------

1	<code>git checkout -b nouvelle-fonctionnalite</code>	Créer une branche
2	<code>echo "Nouvelle fonction" > fonction.txt</code>	Créer un fichier
3	<code>git add . && git commit -m "Ajout fonction"</code>	Committ sur la branche
4	<code>git checkout main</code>	Revenir sur main
5	<code>git merge nouvelle-fonctionnalite</code>	Fusionner
6	<code>git log --oneline --graph</code>	Voir le graphe
7	<code>git branch -d nouvelle-fonctionnalite</code>	Supprimer la branche

Exercice

04

Corriger une erreur

■ Objectif : Apprendre à annuler des modifications et des commits

#	Commande	Explication
1	<code>echo "Erreur ici" > erreur.txt</code>	Créer un fichier avec erreur
2	<code>git add erreur.txt</code>	Préparer
3	<code>git restore --staged erreur.txt</code>	Annuler le git add
4	<code>git restore erreur.txt</code>	Annuler les modifications
5	<code>echo "Bon contenu" > bon.txt</code>	Créer bon fichier
6	<code>git add . && git commit -m "Ajout bon contenu"</code>	Commiter
7	<code>git revert HEAD</code>	Annuler le commit proprement

Exercice

05

Envoyer sur GitHub

■ Objectif : Connecter votre dépôt local à GitHub et envoyer votre code

#	Commande	Explication
2	<code>git remote add origin https://github.com/mon-projet/projet.git</code>	Connecter
3	<code>git remote -v</code>	Vérifier la connexion
4	<code>git push -u origin main</code>	Premier push
5	<code>echo "Mise à jour" >> notes.txt</code>	Modifier un fichier
6	<code>git add . && git commit -m "Mise à jour"</code>	Commiter

7	<code>git push</code>	Push classique
---	-----------------------	----------------